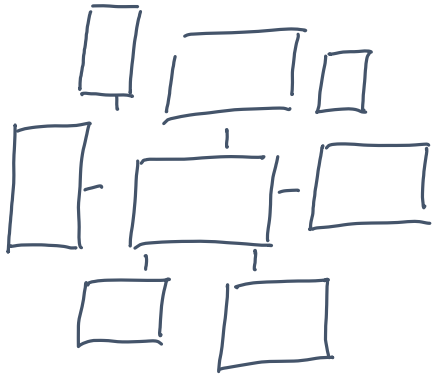




FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

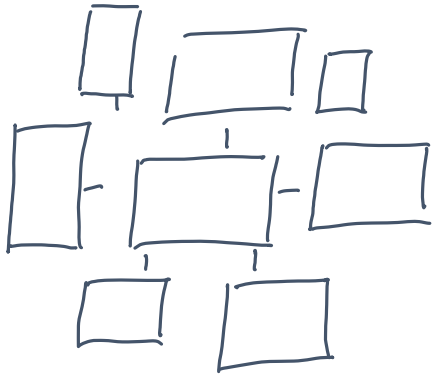
fbeit

Design Verification

Today's Agenda

Intended topics for today's session

- Introduction to SystemVerilog – Basic Testbench Design
- Testbench based Functional Design Simulation using Intel Quartus Prime Lite, SystemVerilog HDL and Mentor ModelSim
- Live Demonstration



FPGA-based SoC Design

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Design Verification

Recommended Readings

Textbooks, Application Notes, White Papers ...

- Sutherland, S., “RTL Modeling with SystemVerilog for Simulation and Synthesis: Using SystemVerilog for ASIC and FPGA Design”, CreateSpace Independent Publishing Platform, 2017
- Spear, C., “SystemVerilog for Verification: A Guide to Learning the Testbench Language Features”, Springer, 3rd edition, 2012.



Introduction to SystemVerilog – Part#5

International Master of Science in Electrical Engineering

Prof. Dr. C. Jakob

University of Applied Sciences Darmstadt

h_da

Faculty of Electrical Engineering and Information Technology

fbeit

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Basic Testbench Design using SystemVerilog HDL

Introduction to SystemVerilog

- Timing control in SystemVerilog HDL testbenches

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Include unit delay times using the # operator ...

```
1. #20                      // Delay the execution of the respective block by 20ns...
2. #(10*CLK_PERIOD)        // ...
```

SystemVerilog Testbench Design - Delay operator

Event control – Rising/Falling Signal Edges

```
1. @(posedge clk)           // event control, wait for the rising edge of the clk
```

SystemVerilog Testbench Design – Event Control

Event control – Signal Levels ...

```
1. wait((state == IDLE) && (start_sys == 1'b1)) // Level sensitive event
                                           // control ...
```

SystemVerilog Testbench Design – Event Control

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Basic Testbench Design using SystemVerilog HDL

Introduction to SystemVerilog

- Tasks

Test Vector Generation

- Very often in testbenches, the same piece of code is repeated several times.
- This is often the case when different sets of inputs are applied.

```
1. task SEND_ACKNOWLEDGE;           // semicolon needed
2.     begin                         // begin-end necessary
3.     // Send an acknowledge
4.     tb_ack = 1;
5.     @(posedge tb_local_clock);
6.     # 1;
7.     tb_ack = 0;
8.     @(posedge tb_local_clock);
9.     # 1;
10.    end                           // begin-end necessary
11. endtask                         // no semicolon
```

SystemVerilog Task

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Basic Testbench Design using SystemVerilog HDL

Introduction to SystemVerilog

- Basic Testbench configuration using Intel Quartus Prime Lite and Mentor ModelSim

Basic Testbench Configuration

using Intel Quartus Prime Lite and Mentor ModelSim

Launching **ModelSim** from Quartus Prime

1. Open the settings dialog box by clicking **Assignment | Settings** menu or by using the keyboard shortcut Ctrl+Shift+E
2. Select the **EDA Tool Settings | Simulation** option in the sidebar
3. Enable testbench by selecting the option **Compile testbench**
4. Click **Testbenches...** button to create testbenches
5. Click **New** button to create new testbench
6. In the **New Testbench Settings** dialog, provide a name for the testbench
7. Specify the top-level module in your testbench file
8. Add the testbench file by clicking the File name ellipsis button
9. Click **Ok** to create a new testbench
10. Afterwards, click **Apply** in Settings dialog box
11. To launch ModelSim simulator, you have to first synthesize your design by clicking the **Start Analysis & Synthesis** icon or by using the Ctrl+K shortcut (Make sure that your design is set as the current top-level entity - **Project | Set as Top-Level entity**)
12. After the Analysis & Synthesis process is successfully completed, launch the simulation by **Tools | Run Simulation | RTL Simulation**

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Basic Testbench Design using SystemVerilog HDL

Introduction to SystemVerilog

- Time Base Generation Core – Using an extended testbench concept for LED simulation purposes

Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

SystemVerilog Testbench ...

```
1. `timescale 1ns/1ps
2. `define HALF_CLOCK_PERIOD    10
3. `define RESET_PERIOD        200
4. `define SIM_DURATION         50000
5.
6. module time_base_generation_tb();
7.     logic tb_q;
8.     // ### clock generation process ...
9.     logic tb_local_clock = 0;
10.    initial
11.        begin: clock_generation_process
12.            tb_local_clock = 0;
13.            forever
14.                #`HALF_CLOCK_PERIOD tb_local_clock = ~tb_local_clock;
15.        end
16.
17.    logic tb_local_reset_n = 0;
```

1/2

[SystemVerilog] Source Code: time_base_generattion_v2_tb.sv

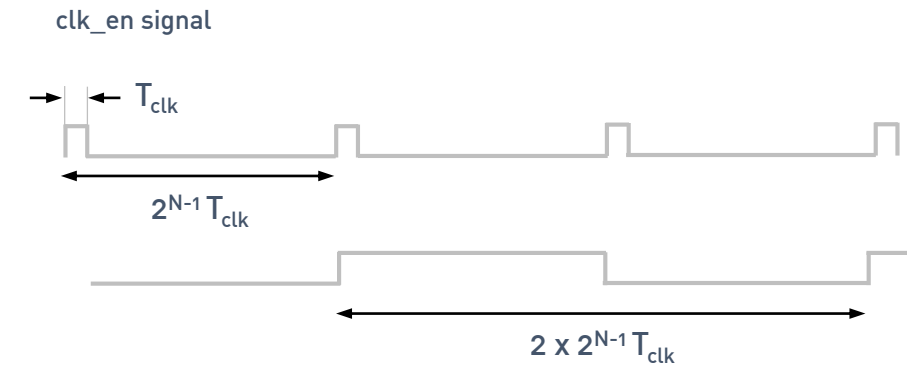
Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

SystemVerilog Testbench ...

```
17. initial
18.     begin: reset_generation_process
19.         $display ("Simulation starts ...");
20.         #`RESET_PERIOD tb_local_reset_n = 1'b1;
21.         #`SIM_DURATION
22.         $stop();
23.     end
24.
25. time_base_generator #(.CLK_CYCLES(1025)) inst_0 (
26.     .clk(tb_local_clock),
27.     .reset_n(tb_local_reset_n),
28.     .q(tb_q)
29. );
30. logic tb_test_led;
31. always_ff@(posedge tb_local_clock)
32.     if(tb_local_reset_n == 1'b0)
33.         tb_test_led <= 0;
34.     else
35.         if(tb_q == 1) tb_test_led <= ~ tb_test_led;
36. endmodule
```

2/2



Introduction to SystemVerilog

h_da – fbeit - FPGA-based SoC Design

Basic Testbench Design using SystemVerilog HDL

Introduction to SystemVerilog

- More useful testbench features ...

4-bit Timer with Integrated Overflow Indicator

```
1. module count_up_4bit_co(  
2.     input logic clk, input logic reset_n,  
3.     output logic [3:0] q,  
4.     output logic q_co  
5. );  
6. always_ff@(posedge clk)  
7.     if(reset_n == 1'b0)  
8.         q <= 4'b0000;  
9.     else  
10.        q <= q + 1'b1;  
11. logic reg_tmp;  
12. always_ff@(posedge clk)  
13.     if(reset_n == 1'b0)  
14.         reg_tmp <= 1'b0;  
15.     else  
16.         reg_tmp <= q[0];  
17.  
18. assign q_co = (q == 4'b0000) ? reg_tmp : 1'b0;  
19.  
20. endmodule
```

1/1

Corresponding Testbench ...

```
1. `timescale 1ns/1ps
2. `define HALF_CLOCK_PERIOD 5
3. `define RESET_PERIOD 20
4. `define SIM_DURATION 1000
5.
6. module count_up_4bit_co_v2_tb();
7.     logic [3:0] tb_q_count; logic tb_q_co;
8.     logic tb_local_clock = 0;
9.     initial
10.         begin: clock_generation_process
11.             tb_local_clock = 0;
12.             forever #`HALF_CLOCK_PERIOD tb_local_clock = ~tb_local_clock;
13.         end
14.     logic tb_local_reset_n = 0;
15.     initial
16.         begin: reset_generation_process
17.             $display ("Simulation starts ...");
18.             #`RESET_PERIOD tb_local_reset_n = 1'b1;
19.             #`SIM_DURATION $stop();
20.         end
```

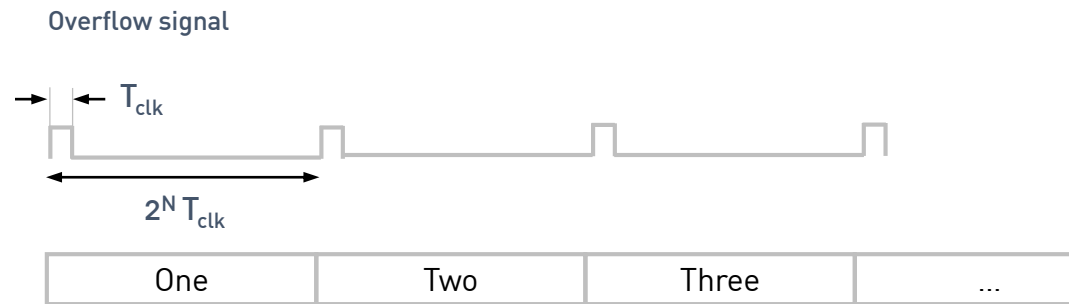
1/2

Corresponding Testbench ...

```
21. count_up_4bit_co inst_dut( .clk(tb_local_clock),  
22.                             .reset_n(tb_local_reset_n),  
23.                             .q(tb_q_count),  
24.                             .q_co(tb_q_co));  
25.  
26. logic [3:0] overflow_count = 0;  
27. string state_string; // ASCII string ...  
28.  
29. always_ff@(posedge tb_q_co) begin  
30.     $display("Overflow detected!\n");  
31.     @(posedge tb_local_clock); @(posedge tb_local_clock); // Two clock delays ...  
32.     overflow_count = overflow_count + 1'b1;  
33.     case(overflow_count)  
34.         4'b0000: state_string = "Zero";  
35.         4'b0001: state_string = "One";  
36.         4'b0010: state_string = "Two";  
37.         4'b0011: state_string = "Three";  
38.         4'b0100: state_string = "Four";  
39.     endcase  
40. end  
41. endmodule
```

2/2

4-bit Timer with Integrated Overflow Indicator



Notes

- A simple way to improve the readability of a timing diagram ...
- In the ModelSim waveform viewer, make sure that the ASCII string is also formatted as an ASCII string ;-)